SQL Analysis for Pizza sales : Unlocking Business Insights

Welcome to our presentation on leveraging SQL to extract valuable business insights from pizza sales data. We'll explore how simple queries can reveal powerful trends, helping us optimize operations and boost revenue.

Agenda

From Data to Decisions

01

The Basics: Foundational Metrics

Understanding core sales figures and popular items.

02

Intermediate Insights: Deeper Dive

Analyzing categories, order patterns, and daily averages.

03

Advanced Analytics: Strategic Growth

Uncovering revenue contributions and time-based trends.

04

Your Role: Interactive Analysis

Empowering you to explore and interpret the data further.

The Basics

Uploading the csv data files in SQL databases

```
create database pizzahut;

create table orders( order_id int not null ,
order_date date not null  ,
order_time time  not null ,
primary key(order_id));

create table order_details( order_details_id int not null ,
pizza_id text not null  ,
order_id int not null ,
quantity int not null ,
primary key(order_details_id));
```

Key Sales Metrics

SQL 1

Total Orders

Count of all transactions.

SQL 2

Total Revenue

Cumulative income from sales.

SQL 3

Highest-Priced Pizza

Identifies our premium offering.

SQL 4

Most Common Size

Reveals customer preference for sizing.

These initial queries (`SELECT COUNT(*)`, `SUM(price)`, `MAX(price)`, and `MODE()` on size) give us immediate insights into our operational scale and customer preferences.

First Query : Retrieve the total numbers of order placed .

```
SELECT
    COUNT(order_id) AS total_orders
FROM
    orders;
```

total_orders
21350

Second Query : Calculate the total revenue generated from pizza sales.

```
SELECT
    ROUND(SUM(order_details.quantity * pizzas.price),
            2) AS total_revenue
FROM
    order_details
        JOIN
    pizzas ON pizzas.pizza_id = order_details.pizza_id
```

Result Grid

total_revenue
817860.05

Third Query : Identify the highest-priced pizza.

```
SELECT
    pizza_types.name, pizzas.price
FROM
    pizza_types
        JOIN
    pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
ORDER BY pizzas.price DESC
LIMIT 1;
```

Result Grid | Filter R

name	price
The Greek Pizza	35.95

Fourth Query : Identify the most common pizza size ordered

```
SELECT
    pizzas.size,
    COUNT(order_details.order_details_id) AS order_count
FROM
    pizzas
        JOIN
    order_details ON pizzas.pizza_id = order_details.pizza_id
GROUP BY pizzas.size
ORDER BY order_count DESC;
```

Result Grid | Filter

size	order_count
L	18526
M	15385
S	14137
XL	544
XXL	28

Fifth Query : List the top 5 most ordered pizza types along with their quantities.

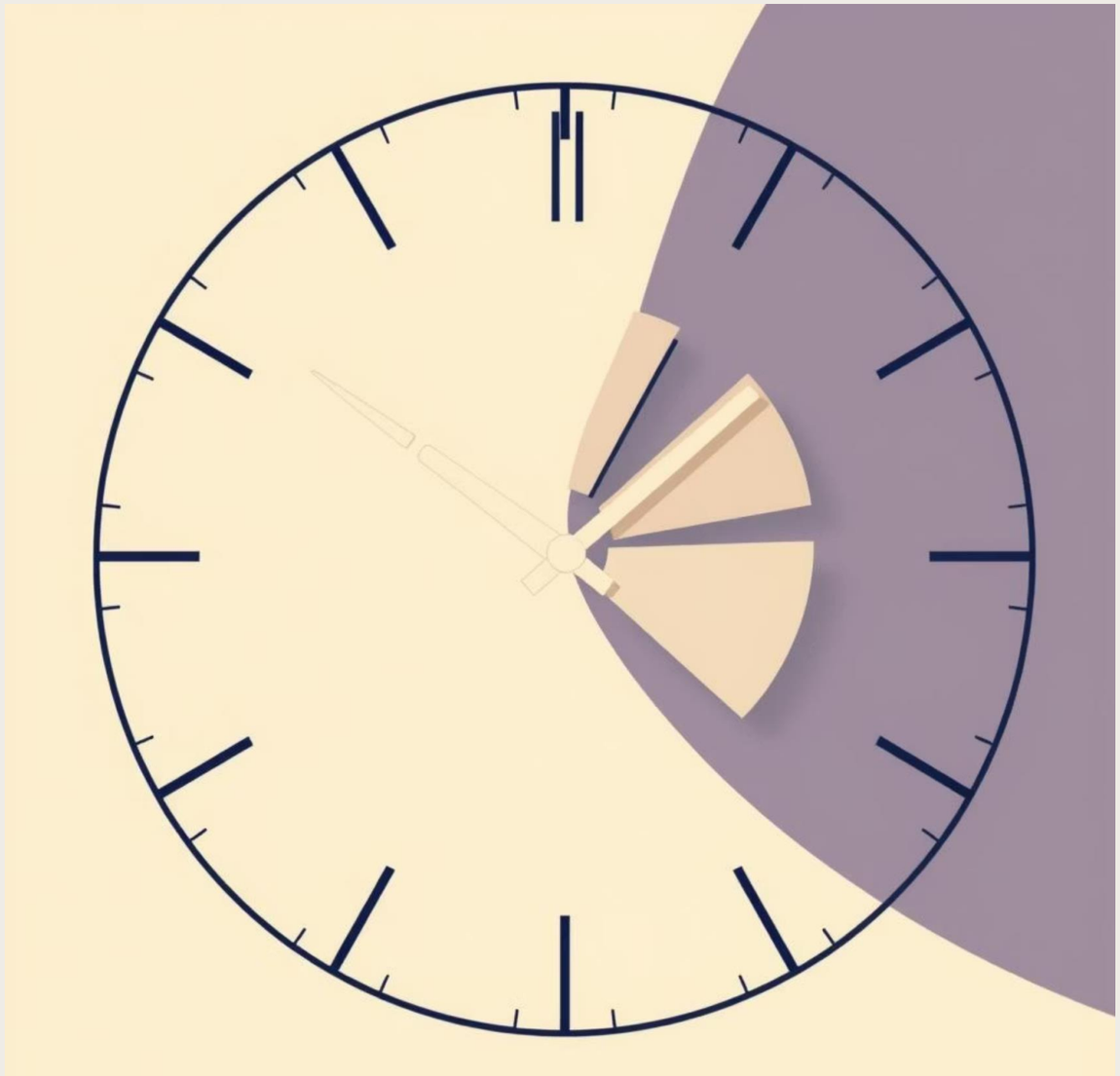
```sql
SELECT
    pizza_types.name,
    SUM(order_details.quantity) AS total_quantity
FROM
    pizza_types
        JOIN
    pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
        JOIN
    order_details ON order_details.pizza_id = pizzas.pizza_id
GROUP BY pizza_types.name
ORDER BY total_quantity DESC
LIMIT 5;
```

Result Grid | Filter Rows:

name	total_quantity
The Classic Deluxe Pizza	2453
The Barbecue Chicken Pizza	2432
The Hawaiian Pizza	2422
The Pepperoni Pizza	2418
The Thai Chicken Pizza	2371

Chapter 2: Intermediate Insights

Deeper Dive into Operations



Moving beyond simple counts, intermediate queries allow us to connect different data points and uncover more nuanced patterns. This helps us optimize staffing, ingredient sourcing, and menu design.

- Category-wise distribution
- Hourly order patterns
- Average daily orders

Order Distribution by Hour

Understanding when customers order helps us streamline operations, optimize staffing, and even tailor promotions for specific times of the day. This requires extracting the hour from timestamps and grouping accordingly.

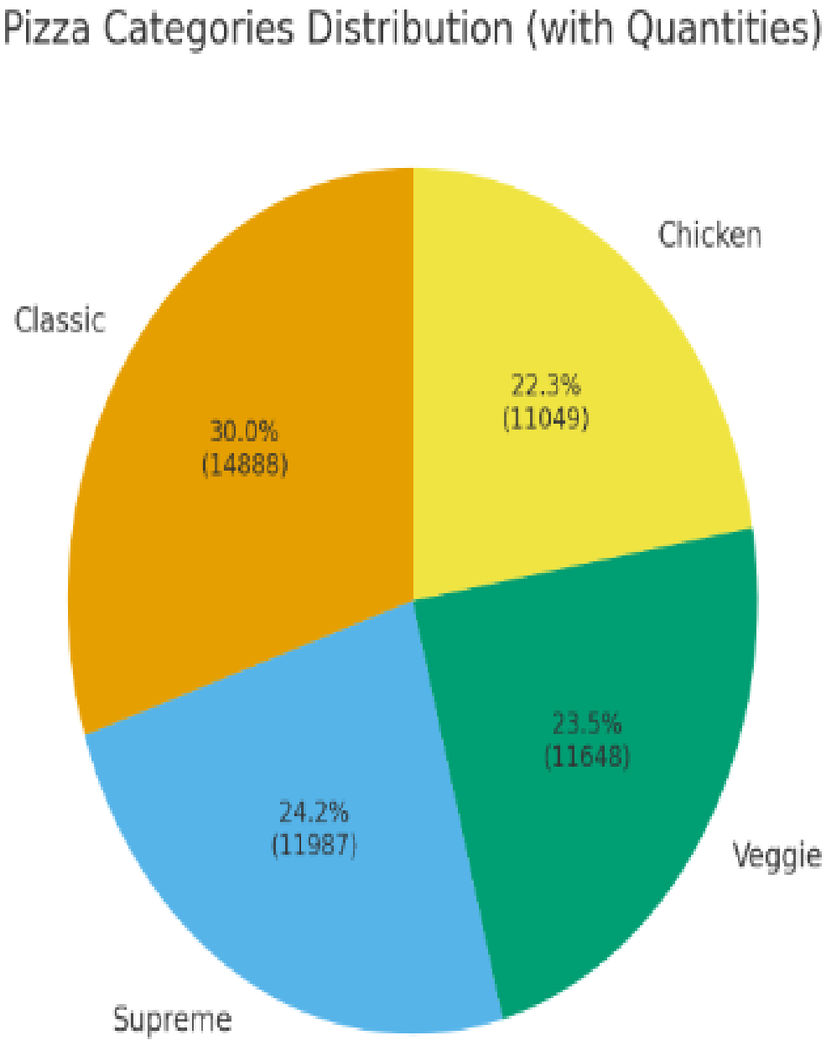
This visual highlights our busiest periods, essential for efficient kitchen management and delivery services. Early afternoon and evening consistently show higher order volumes

Sixth Query : Join the necessary tables to find the total quantity of each pizza category ordered.

```sql
SELECT
    pizza_types.category,
    SUM(order_details.quantity) AS quantity
FROM
    pizza_types
        JOIN
    pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
        JOIN
    order_details ON order_details.pizza_id = pizzas.pizza_id
GROUP BY pizza_types.category
ORDER BY quantity DESC;
```

Result Grid | Filte

category	quantity
Classic	14888
Supreme	11987
Veggie	11649
Chicken	11050

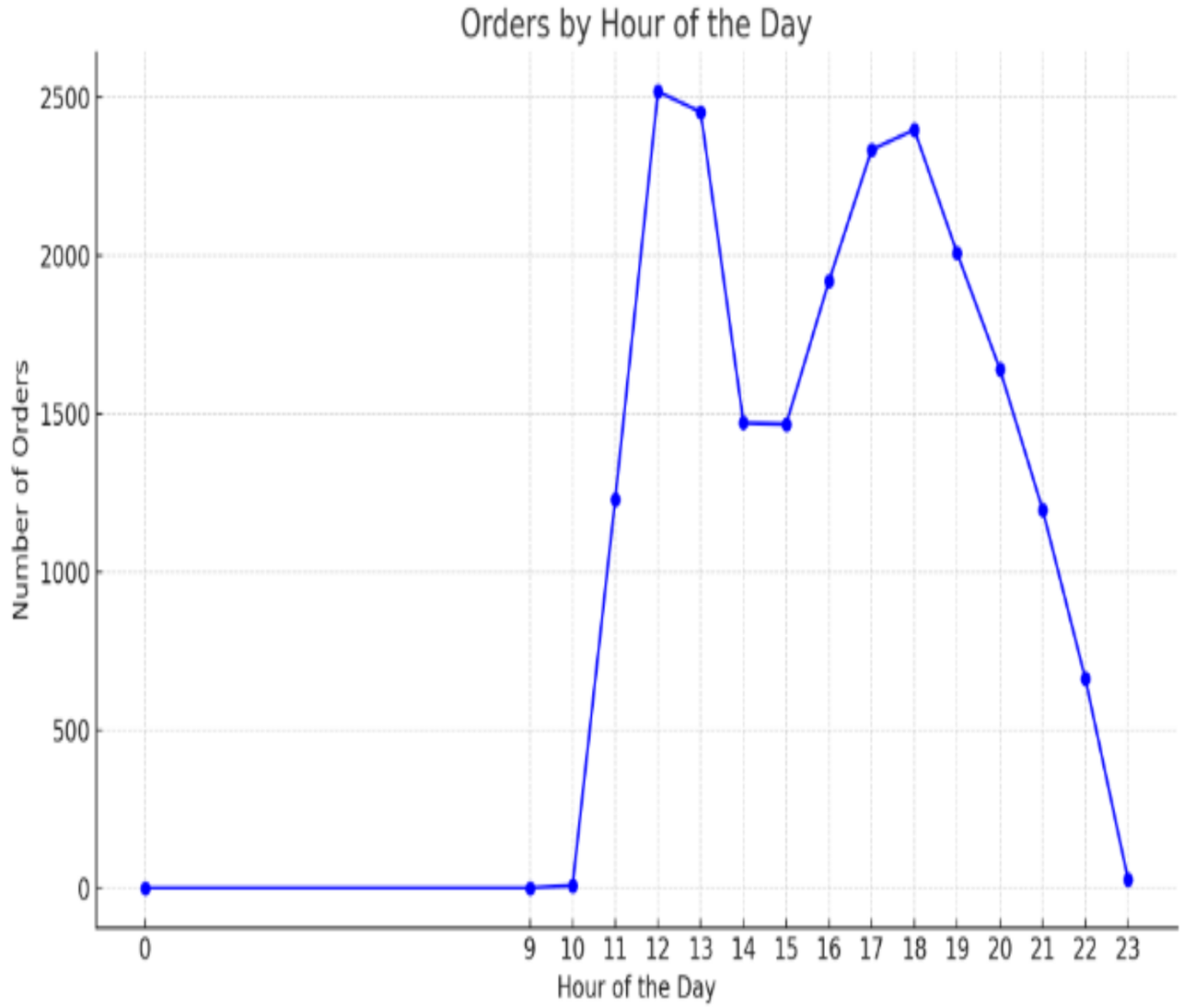


Seventh Query : Determine the distribution of orders by hour of the day .

```sql
SELECT
    HOUR(order_time) AS hour, COUNT(order_id)
FROM
    orders AS order_count
GROUP BY HOUR(order_time);
```

Result Grid | Filter Rows:

hour	COUNT(order_id)
11	1231
12	2520
13	2455
14	1472
15	1468
16	1920
17	2336
18	2399
19	2009
20	1642
21	1198
22	663
23	28
10	8
9	1



Eighth Query : Join relevant tables to find the category-wise distribution of pizzas.

```
select category , count(name) from pizza_types
group by category
```

Result Grid | Filter Rows:

category	count(name)
Chicken	6
Classic	8
Supreme	9
Veggie	9

Ninth Query : Group the orders by date and calculate the average number of pizzas ordered per day.

```
SELECT
    round(AVG(daily_pizzas) , 0) AS avg_pizzas_per_day
FROM
    (SELECT
        orders.order_date,
            SUM(order_details.quantity) AS daily_pizzas
    FROM
        orders
    JOIN order_details ON orders.order_id = order_details.order_id
    GROUP BY orders.order_date) AS daily_summary;
```

Result Grid | Filter Rows

avg_pizzas_per_day
138

Chapter 3: Advanced Analytics

Strategic Growth & Revenue Optimization

These advanced analyses reveal the strategic impact of each pizza type and how revenue evolves over time. This information is vital for long-term planning, menu adjustments, and maximizing profitability.

- Percentage contribution to total revenue
- Cumulative revenue trends
- Top pizza types by revenue per category

Tenth Query : Determine the top 3 most ordered pizza types based on revenue.

```sql
SELECT
    pizza_types.name,
    SUM(order_details.quantity * pizzas.price) AS revenue
FROM
    pizza_types
        JOIN
    pizzas ON pizzas.pizza_type_id = pizza_types.pizza_type_id
        JOIN
    order_details ON order_details.pizza_id = pizzas.pizza_id
GROUP BY pizza_types.name
ORDER BY revenue DESC
LIMIT 3;
```

Result Grid | Filter Rows:

name	revenue
The Thai Chicken Pizza	43434.25
The Barbecue Chicken Pizza	42768
The California Chicken Pizza	41409.5

Eleventh Query : Calculate the percentage contribution of each pizza type to total revenue.

```sql
SELECT
    pizza_types.name,
    ROUND(
        (SUM(order_details.quantity * pizzas.price) /
        (SELECT Round(SUM(order_details.quantity * pizzas.price),2) as total_sales
         FROM order_details
         JOIN pizzas ON order_details.pizza_id = pizzas.pizza_id)
        ) * 100, 2
    ) AS revenue_percentage
FROM pizza_types
JOIN pizzas
  ON pizzas.pizza_type_id = pizza_types.pizza_type_id
JOIN order_details
  ON order_details.pizza_id = pizzas.pizza_id
GROUP BY pizza_types.name
ORDER BY revenue_percentage DESC;
```

Result Grid | Filter Rows:

name	revenue_percentage
The Thai Chicken Pizza	5.31
The Barbecue Chicken Pizza	5.23
The California Chicken Pizza	5.06
The Classic Deluxe Pizza	4.67
The Spicy Italian Pizza	4.26
The Southwest Chicken Pizza	4.24
The Italian Supreme Pizza	4.09
The Hawaiian Pizza	3.95
The Four Cheese Pizza	3.95
The Sicilian Pizza	3.78
The Pepperoni Pizza	3.69
The Greek Pizza	3.48
The Mexicana Pizza	3.27
The Five Cheese Pizza	3.19
The Pepper Salami Pizza	3.12

Result Grid | Filter Rows: | Export:

name	revenue_percentage
The Italian Capocollo Pizza	3.07
The Vegetables + Vegetable...	2.98
The Prosciutto and Arugula ...	2.96
The Napolitana Pizza	2.95
The Spinach and Feta Pizza	2.85
The Big Meat Pizza	2.81
The Pepperoni, Mushroom, ...	2.3
The Chicken Alfredo Pizza	2.07
The Chicken Pesto Pizza	2.04
The Soppressata Pizza	2.01
The Italian Vegetables Pizza	1.96
The Calabrese Pizza	1.95
The Spinach Pesto Pizza	1.91
The Mediterranean Pizza	1.88
The Spinach Supreme Pizza	1.87
The Spinach Supreme Pizza	1.87
The Green Garden Pizza	1.71
The Brie Carre Pizza	1.42

Twelvth Query : Analyze the cumulative revenue generated over time

```sql
select order_date ,
sum(revenue) over (order by order_date) as cum_revenue
from
(select orders.order_date,
sum(order_details.quantity * pizzas.price) as revenue
from order_details join pizzas
on order_details.pizza_id = pizzas.pizza_id
join orders
on orders.order_id = order_details.order_id
group by orders.order_date) as sales ;
```

Result Grid | Filter Rows: | Expo

order_date	cum_revenue
2015-01-01	2713.8500000000004
2015-01-02	5445.75
2015-01-03	8108.15
2015-01-04	9863.6
2015-01-05	11929.55
2015-01-06	14358.5
2015-01-07	16560.7
2015-01-08	19399.05
2015-01-09	21526.4
2015-01-10	23990.350000000002
2015-01-11	25862.65
2015-01-12	27781.7
2015-01-13	29831.300000000003
2015-01-14	32358.700000000004
2015-01-15	34343.50000000001

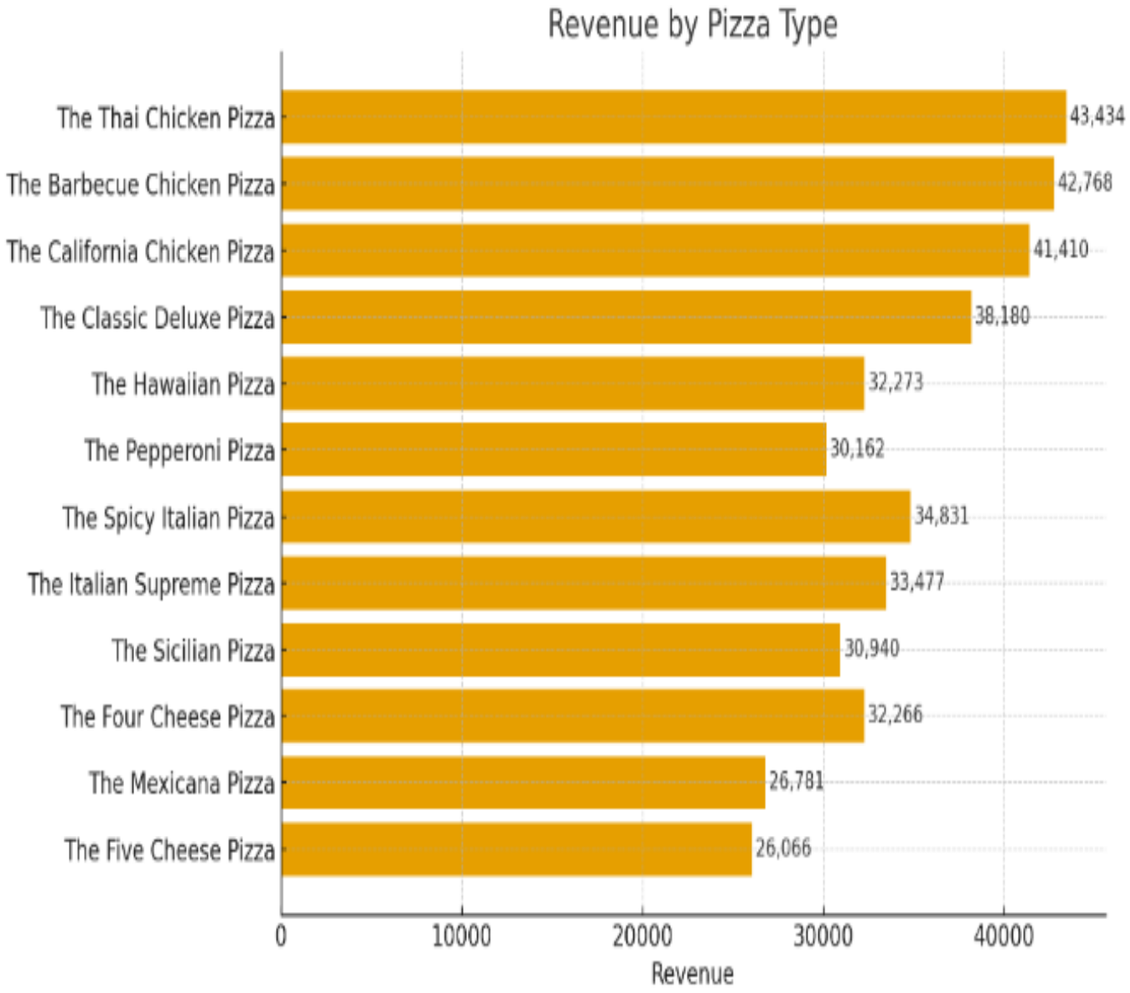
Thirteenth :Determine the top 3 most ordered pizza types based on revenue for each pizza category.

```sql
select name , revenue from

(select  category , name , revenue ,
rank() over(partition by category order by revenue desc ) as rn
from
(select pizza_types.category , pizza_types.name,
sum((order_details.quantity)* pizzas.price) as revenue
from pizza_types join pizzas
on pizza_types.pizza_type_id = pizzas.pizza_type_id
join order_details
on order_details.pizza_id = pizzas.pizza_id
group by pizza_types.category , pizza_types.name) as  a) as b
where rn<= 3 ;
```

Result Grid | Filter Rows:

name	revenue
The Thai Chicken Pizza	43434.25
The Barbecue Chicken Pizza	42768
The California Chicken Pizza	41409.5
The Classic Deluxe Pizza	38180.5
The Hawaiian Pizza	32273.25
The Pepperoni Pizza	30161.75
The Spicy Italian Pizza	34831.25
The Italian Supreme Pizza	33476.75
The Sicilian Pizza	30940.5
The Four Cheese Pizza	32265.70000000065
The Mexicana Pizza	26780.75
The Five Cheese Pizza	26066.5



Pizza Sales Analysis Summary

In this analysis, I explored the sales data of different pizza types to understand their **revenue performance**.

✅ Steps I Performed:

1 . Data Extraction using SQL

- Queried the database to calculate the total revenue for each pizza type.
- Revenue was calculated as:

$$\text{Revenue} = \textit{Quantity Ordered} \times \textit{Price per Pizza}$$

2 . Ranking Pizza Types

- Aggregated sales by pizza type.
- Identified the **top-performing pizzas** based on revenue contribution.

3 . Visualization

- Created a **bar chart** to show the revenue of each pizza type.
- This helps in quickly identifying which pizzas are the best-sellers and which ones contribute less.

📌 Key Insights:

Top Revenue Generator: *The Thai Chicken Pizza* generated the highest revenue (₹43,434 approx.).
Other strong performers include *Barbecue Chicken Pizza* and *California Chicken Pizza*.
Lower-performing pizzas include *The Mexicana Pizza* and *The Five Cheese Pizza*, which contributed the least revenue.

🎯 Why This Analysis Matters:

Helps managers identify **customer preferences**.
Supports **menu planning** (promoting popular pizzas and rethinking weaker ones).
Useful for **marketing strategies** (e.g., bundle offers with top sellers).

Key Takeaways & Next Steps

Key Takeaways

- **SQL is a Powerful Tool:** Transform raw data into actionable business intelligence.
- **Identify Peak Hours:** Optimize staffing and resource allocation for efficiency.
- **Understand Customer Preferences:** Tailor your menu and promotions based on popular items and sizes.
- **Strategic Revenue Growth:** Pinpoint which pizzas and categories drive the most profit.

Your Interactive Report

The SQL queries provided offer a foundation. Now, it's your turn to explore:

- **Run the Queries:** Execute the provided SQL to verify results.
- **Analyze Further:** Experiment with different groupings or filters.
- **Visualize Trends:** Use your favorite BI tools to create more interactive dashboards.
- **Share Insights:** Collaborate with your teams to make data-driven decisions.